



OPEN SOURCE ENGINEERING

Student ID: 2400030538

Semester: Odd

Academic Year: 2025-2026

Course Code: 24CS02EF

Under the guidance of

Dr. Sripath Roy Koganti

1 Understanding the Core Ubuntu Linux Distribution

1.0.1 1. Overview and Philosophy

Ubuntu is a powerful, free, and open-source operating system built upon the stable foundation of Debian Linux. It stands as the world's most popular Linux distribution for desktop use, successfully blending cutting-edge features with unparalleled user-friendliness. Developed and maintained by Canonical Ltd., Ubuntu's guiding principle is "Linux for human beings." This philosophy drives its commitment to accessibility, stability, and providing an intuitive computing experience for everyone, from novice users to seasoned developers.

1.0.2 2. The Desktop Experience (GNOME)

The standard Ubuntu desktop utilizes the **GNOME** desktop environment, which presents a modern, clean, and highly efficient graphical interface. The key design elements include a permanent dock (launcher) on the left side for quick access to essential applications, and the **Activities Overview**. This view, easily accessed by pressing the Super (Windows) key, provides a centralized hub for managing all open windows, workspaces, and system-wide searching. This streamlined workflow makes Ubuntu feel contemporary and ensures high productivity. Furthermore, Ubuntu is recognized for its strong, out-of-the-box hardware detection and compatibility, simplifying the setup process for most users.

1.0.3 3. Software Management and Packaging

Ubuntu employs a robust dual-system for software management. The traditional and reliable **Advanced Packaging Tool (APT)** manages **DEB** packages, handling core system utilities and standard applications sourced from official repositories. Complementing this is the use of **Snaps**, a modern, containerized package format pioneered by Canonical. Snaps bundle an application with all its required dependencies, guaranteeing consistent performance across different Ubuntu versions. Crucially, Snaps run in a **sandboxed** environment, isolating them from the rest of the operating system to significantly enhance overall application security. This flexibility ensures users have access to a vast, up-to-date, and secure software library.

2 Encryption and GPG

2.1 Types of Encryption in Ubuntu

Ubuntu offers two primary approaches to encryption: **Full Disk Encryption (FDE)** and **File/Directory Encryption**.

2.1.1 1. Full Disk Encryption (FDE)

- **What it is:** FDE encrypts the entire hard drive or a large partition, including the operating system files, swap space, and user directories.
- **How it works:** Ubuntu uses **LUKS** (Linux Unified Key Setup) for FDE. When the system boots, you are prompted for a **passphrase**. If correct, LUKS decrypts the entire drive, and the decryption process runs transparently in the background while the system is in use.
- **Purpose:** The primary defense against data loss due to **theft** or **physical access** to the computer when it is turned off. If someone steals the hard drive, the data is useless without the LUKS passphrase.
- **Implementation:** FDE is typically enabled during the Ubuntu installation process by selecting the "Encrypt the new Ubuntu installation" option. It's much more difficult to enable after installation.

2.1.2 2. File and Directory Encryption

- **What it is:** This method encrypts specific files, directories, or messages, offering granular control over which data is protected.
- **Tools:**
 - **GPG (GNU Privacy Guard):** The standard, used for encrypting individual files and especially for secure communication using **public-key cryptography**.
 - **eCryptfs (older):** Previously used for encrypting the user's Home directory, but has been largely phased out for FDE.

2.2 GPG (GNU Privacy Guard) Explained

GPG is the GNU implementation of the **OpenPGP** standard (originally Pretty Good Privacy - PGP). It is essential for protecting individual files and ensuring secure, authenticated communication.

2.2.1 1. Core GPG Concepts

GPG relies on **asymmetric cryptography**, which uses a pair of mathematically linked keys:

- **Public Key:** This key is shared with everyone. It can be used to **encrypt** a message that only you can read, or to **verify** a signature you created.
- **Private (Secret) Key:** This key is kept **secret** and is protected by a strong passphrase. It is used to **decrypt** messages sent to you, or to **digitally sign** files to prove they came from you.

2.2.2 2. Basic GPG Command-Line Usage

GPG is usually pre-installed on Ubuntu and is primarily used through the command line (Terminal).

A. Generating a Key Pair The first step is to create your public and private key pair:

Bash

```
gpg --full-generate-key
```

You will be prompted to select the key type (RSA and RSA is common), keysize (4096 is recommended), expiration date, and your Real Name, Email, and a strong **passphrase** to protect your private key.

B. Encrypting a File for Yourself (Symmetric Encryption) To quickly encrypt a file using a single passphrase (like a standard password), use symmetric encryption:

Bash

```
gpg -c myfile.txt
```

This command will prompt you for a passphrase and create an encrypted file named `myfile.txt.gpg`.

C. Encrypting a File for Someone Else (Asymmetric Encryption) To securely send a file, you must use the recipient's **Public Key** (which you must have previously imported into your keyring with `gpg --import`):

Bash

```
gpg --encrypt --recipient "recipient@example.com" mysecretfile.doc
```

This creates `mysecretfile.doc.gpg`. Only the recipient, who holds the corresponding Private Key, can decrypt it.

D. Decrypting a File To decrypt a file that was encrypted for you:

Bash

```
gpg --decrypt mysecretfile.doc.gpg
```

You will be prompted for the passphrase that protects your Private Key. You can use the `--output` option to specify the decrypted

3 Sending Encrypted Email

3.1 Prerequisite: Setting Up GPG

Before you can send or receive encrypted mail, both you and your recipient must have GPG keys set up and exchanged:

1. **Generate Keys:** Both parties must have generated a public/private key pair using GPG (as discussed previously, using `gpg --full-generate-key`).
2. **Exchange Public Keys:** You need the recipient's **Public Key**, and they need your Public Key. You can exchange these by:
 - **Exporting** the key: `gpg --armor --export 'Recipient Name' > recipient_key.asc` and sending the `.asc` file.
 - **Uploading** the key to a public key server.
3. **Import Key:** You must import the recipient's key into your GPG keyring: `gpg --import recipient_key.asc`.

3.2 Sending the Encrypted Email

The most common and user-friendly way to send GPG-encrypted emails on Ubuntu is by using **Mozilla Thunderbird** with the **Enigmail** add-on (or its built-in equivalent in modern versions of Thunderbird).

3.2.1 1. Compose the Message

- **Open Thunderbird** and start composing a new email.
- Write your message as usual.

3.2.2 2. Encryption and Signing

You will use the GPG function built into the mail client to perform two critical steps:

1. **Encryption:** You must encrypt the email using the **recipient's Public Key**. Only their corresponding **Private Key** can decrypt it. If you have multiple recipients, you must encrypt the message using the Public Key of *every single recipient*.
2. **Digital Signature:** You **sign** the email using **your Private Key**. This allows the recipient to verify that the email truly came from you and has not been tampered with in transit.

In Thunderbird, this is typically done by clicking a dedicated **OpenPGP or Security** menu or button within the compose window and ensuring both the "**Encrypt**" and "**Sign**" options are checked.

3.2.3 3. Verification and Sending

- The client will check that you have the required **Public Key** for the recipient(s). If a key is missing, it will warn you.
- When you click **Send**, Thunderbird uses GPG to encrypt the message body and attach your digital signature before transmitting the scrambled data.

3.2.4 4. Recipient's Experience (Decryption)

1. The recipient receives the scrambled email.
2. Their email client automatically uses their **Private Key** (protected by their passphrase) to decrypt the message contents, revealing the original text.
3. Their client simultaneously uses your **Public Key** to verify the digital signature, confirming the email's authenticity.

4 Privacy Tools From Prism Break

4.0.1 1. Tor Browser (Web Browsers / Anonymizing Networks)

- **What it is:** A web browser built on Firefox that routes your internet traffic through the Tor network, a volunteer-operated network of relays.
- **Privacy Focus:** Provides **strong anonymity** by obscuring your IP address and location from the websites you visit. It also includes anti-fingerprinting measures.

- **PRISM Break Note:** PRISM Break strongly recommends using Tor Browser for all web surfing when maximum anonymity is required.

4.0.2 2. Debian (Operating Systems)

- **What it is:** A popular and highly ethical GNU/Linux distribution known for its strict adherence to Free Software principles and ethical manifesto.
- **Privacy Focus:** Unlike proprietary operating systems like Windows and macOS (which PRISM Break generally avoids), Debian is fully open-source, allowing for audits. It has a long tradition of software freedom and transparency.
- **PRISM Break Note:** It's recommended as a top GNU/Linux choice for users transitioning from proprietary systems, highlighting its commitment to free software and its stable nature.

4.0.3 3. Thunderbird (Email Clients)

- **What it is:** A free, open-source, and cross-platform email client developed by Mozilla.
- **Privacy Focus:** Thunderbird is the top choice for desktop email due to its open-source nature and its long-standing **native support for OpenPGP** (GPG) encryption and digital signatures. This allows users to easily encrypt and authenticate their emails end-to-end.
- **PRISM Break Note:** It is highly recommended for securely managing email with built-in PGP features.

4.0.4 4. KeePassXC (Password Managers)

- **What it is:** A free, open-source, and cross-platform password manager.
- **Privacy Focus:** It stores all your passwords in a single, highly encrypted database file that is stored **locally** on your device, giving you total control over your sensitive data. It does not rely on a cloud service.
- **PRISM Break Note:** It is preferred for its strong encryption, open-source license, and local-only storage, minimizing exposure to third-party services.

4.0.5 5. Firefox (Web Browsers)

- **What it is:** A fast, flexible, and secure web browser developed by the non-profit Mozilla Foundation.

- **Privacy Focus:** Firefox is open-source and provides extensive privacy controls, including enhanced tracking protection (ETP), container technology, and a robust add-on ecosystem for further hardening security (like uBlock Origin).
- **PRISM Break Note:** While Tor Browser is for anonymity, Firefox is the recommended alternative for general web use when a site doesn't work well with Tor, provided the user configures its settings and replaces the default search engine with a privacy-focused one.

5 Open Source License

Certainly. Here is the information about the **MIT License** organized into clear, descriptive headings, strictly maintaining a paragraph-only format within each section.

5.1 The Core Purpose and Classification

The MIT License is renowned as one of the most permissive and concise open-source licenses currently in use. Originating from the Massachusetts Institute of Technology, its primary goal is to encourage maximum adoption and reuse of software with minimal legal friction. It is formally classified as a **permissive license**, meaning it grants users broad rights to use, modify, and distribute the software without imposing the reciprocal sharing obligations seen in copyleft licenses, such as the GNU General Public License (GPL). This makes the MIT License highly favorable for both commercial enterprises and proprietary software development.

5.2 Granted Rights and Permissions

The license grants blanket permission to any individual or entity obtaining a copy of the software and its associated documentation to deal with the Software without restriction. Specifically, users are granted explicit rights to **use, copy, modify, merge, publish, distribute, sublicense, and/or sell** copies of the software. This expansive grant allows developers to incorporate MIT-licensed code into projects that may ultimately be closed-source and sold commercially, provided they meet the few mandated conditions.

5.3 The Only Two Conditions for Distribution

Unlike licenses that enforce reciprocal sharing, the MIT License has only two critical requirements that must be met when the software is distributed or included in a larger work. The first condition is the mandatory inclusion of the original **Copyright Notice** (e.g., **Copyright <YEAR> <COPYRIGHT HOLDER>**).

The second is the mandatory inclusion of the full **License Text** itself. If these two simple requirements are satisfied, the user can otherwise treat the code as they wish, including releasing their modifications under a proprietary license.

5.4 Disclaimer of Warranty and Liability

A key component of the MIT License is its comprehensive liability disclaimer, which serves to protect the original authors. The license emphatically states that the software is provided **”AS IS,”** meaning it comes without any guarantee or warranty of any kind, whether express or implied, including warranties of merchantability or fitness for a particular purpose. Furthermore, the license explicitly protects the authors and copyright holders, asserting they **shall not be held liable** for any claim, damages, or other liability arising from the use or other dealings in the software. This places the entire risk associated with the software onto the end-user.

6 Piwigo photo gallery

6.1 About

Piwigo Photo Gallery is an open-source, self-hosted photo management and sharing platform designed to provide a clean, powerful, and scalable solution for organizing large image collections. Built with flexibility and long-term maintainability in mind, it offers an efficient way to store, manage, and display photos while giving users complete control over their data. Licensed under the GPL, Piwigo emphasizes openness, community-driven development, and secure image hosting across personal, organizational, and enterprise environments.

6.1.1 Key Features

- **organized photo managemant:** Uses a structured collection system with categories, sub-albums, tags, and metadata, making it easy to manage both small and extremely large libraries.
- **Smart Album Structure:** Offers hierarchical albums, virtual albums, batch processing, and advanced search tools, enabling quick navigation and flexible organization of photos.
- **Elegant User Interface:** Provides a modern, customizable design with support for multiple themes, responsive layouts, and user-friendly navigation to enhance the browsing experience.

- **Advanced Upload Editing Tools:** Allows bulk uploading via web, mobile, FTP, or plugins, and includes features like automatic image resizing, watermarking, metadata extraction, and batch editing.
- **User Permission Control:** Supports fine-grained access management: public galleries, private albums, user groups, role-based permissions, and guest access options.
- **Secure Reliable:** Includes strong security practices, regular community updates, session control, and optional multi-user authentication for secure photo hosting.

6.2 Installation Process (Docker Compose)

The recommended and most convenient method for self-hosting Piwigo Photo Gallery is by using Docker Compose, which efficiently manages both the Piwigo application and its required MariaDB/MySQL database as a unified, multi-container service. Before beginning the deployment, ensure that two prerequisites are met: Docker must be installed on your server or hosting machine, and Docker Compose must be available (it typically comes bundled with Docker Desktop or can be installed independently).

The installation process starts by obtaining the required Docker configuration file. You first navigate to your preferred deployment directory and download the official docker-compose.yml template using a curl command. After downloading, the configuration file needs essential adjustments to ensure proper setup and secure operation. You must edit the environment variables inside the docker-compose.yml file: set the `PIWIGO_URL` to your domain or public IP (e.g., `http://mygallery.com`); define strong, randomly generated passwords for both the database

Once the environment variables are configured, you can proceed to launch the services. Running the command `docker compose up -d` instructs Docker Compose to pull the necessary images and start both the Piwigo and MariaDB containers in detached mode (-d). After the services start successfully, the Piwigo web interface becomes accessible through the URL previously defined in your `PIWIGO_URL`. For local deployments, the default access point is typically `http://localhost:8080`.

Upon first access, you are guided through Piwigo's initial setup wizard where you specify the admin username, password, and the database connection details you configured earlier. Once the setup wizard is completed, your self-hosted Piwigo instance is ready to use, allowing you to begin uploading and organizing your photo collection.

Piwigo Photo Gallery Resources

Translated Document

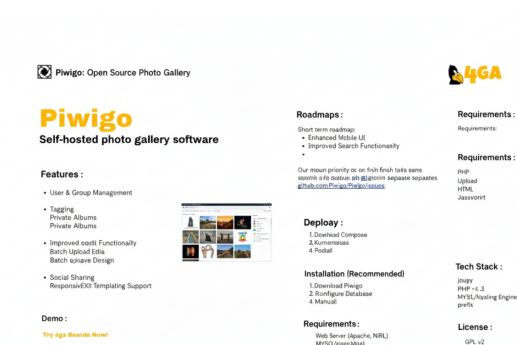


Figure 1: piwigo

Enter Caption

7 Open Source Contribution

7.1 PR 1 : First Contribution

7.1.1 Goal

The project's objective is to help newcomers understand the basic open-source workflow by allowing them to make a simple contribution to the Piwigo ecosystem. The goal of this task is for beginners to add their name to the project's Contributors.md file, making their first successful contribution.

7.1.2 The Contribution Workflow

The tutorial explains the essential fork → clone → edit → pull request sequence, which forms the foundation of collaborative development in most open-source communities, including Piwigo.

7.1.3 1. Setup

- **Fork:**Create a personal copy of the Piwigo-related repository (plugin/theme/documentation) under your own GitHub account.
- **Clone:**Download the forked repository to your local system using the git clone command with the SSH or HTTPS URL.
- **Prerequisites:**Ensure Git is installed on your machine. For users uncomfortable with command-line operations, beginner-friendly GUI clients (GitHub Desktop, GitKraken, Sourcetree) are

recommended.

7.1.4 2. Making Changes

- **Branch:** Create a new isolated branch for your changes using `git switch -c your-new-branch-name`.
- **Edit:** Add your name to the `Contributors.md` file using a text editor.
- **Commit:** Stage the changes with `git add Contributors.md` and save them locally with `git commit -m "Add your-name to Contributors list"`.

7.1.5 3. Submission

- **Push:** Upload your local branch to your GitHub fork using `git push -u origin your-branch-name`.
- **Pull Request (PR):** Go to your GitHub repository and submit a PR via the "Compare & pull request" button for review by the project maintainers.

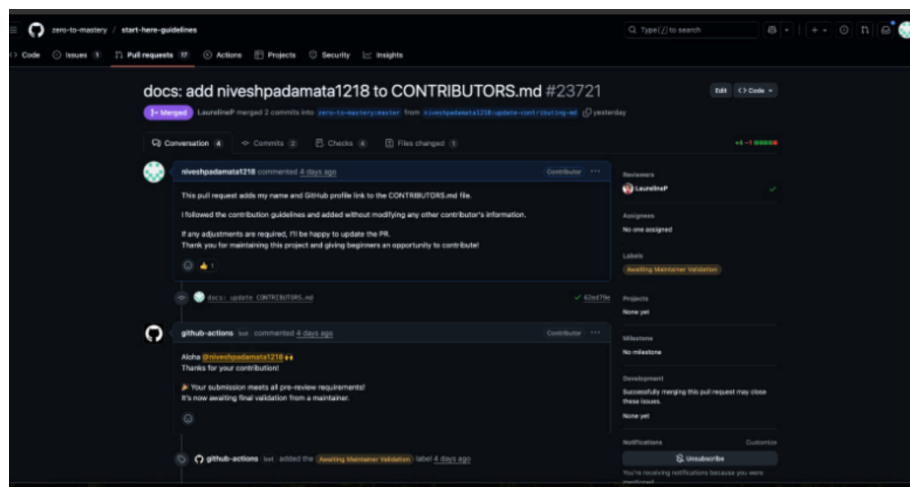
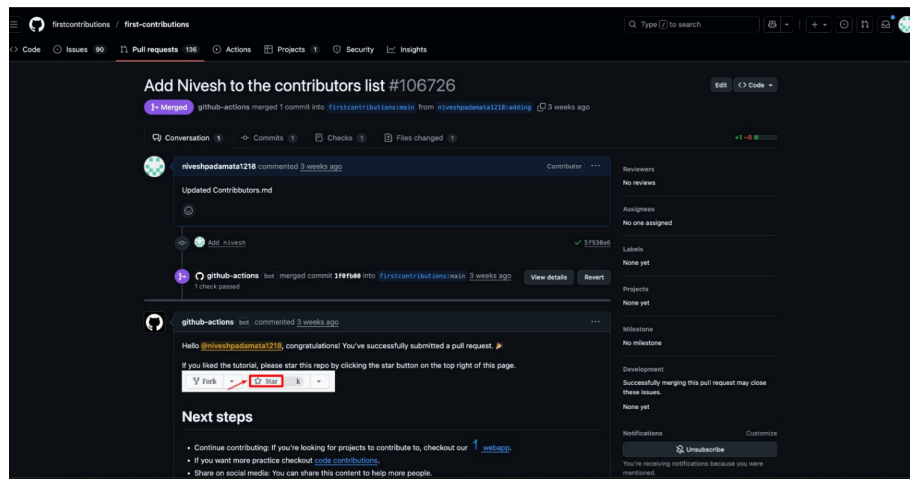
7.1.6 Difficulties and Solutions

The guide anticipates and solves two common beginner issues:

- **Old Git Version:** If the `git switch` command fails, use the older command: `git checkout -b your-new-branch`.
- **Authentication Error:** If `git push` fails due to GitHub removing password support, the solution is to configure an **SSH key** or a **Personal Access Token** and ensure your remote URL is set to the **SSH protocol** (`git remote set-url origin git@github.com:...`).

7.1.7 Next Steps

Upon merging the PR, the user is encouraged to celebrate their first contribution and seek out other beginner-friendly issues on the project list.



7.2 PR 2 :piwigo

Piwigo is a powerful, free, and open-source photo gallery platform designed for individuals, photographers, and organizations who need a reliable and highly customizable system for managing and showcasing large image collections. It is optimized for performance, scalability, and long-term data preservation, making it suitable for personal use as well as professional environments.

Piwigo serves multiple important purposes: it acts as a secure, self-hosted photo management system that gives users complete control over their images; as a highly extensible platform with a rich ecosystem of plugins and themes; and as a flexible gallery solution for sharing albums, managing user permissions, and organizing thousands of photos efficiently. It stands as a robust, feature-rich alternative to proprietary cloud photo services, built on the principles of open-source transparency,

community collaboration, and data ownership.

7.2.1 Licensing and Self-Hosting Options

The entire Piwigo project is distributed under the GNU General Public License (GPL). This license ensures that the software remains fully open-source, requiring that any modified versions of the program that are distributed must also make their corresponding source code available under the same license. This promotes transparency, community collaboration, and long-term freedom for users.

For deployment, Piwigo strongly supports self-hosting, offering multiple reliable installation approaches based on user preference and server environment. The platform can be installed through traditional methods such as manual setup with PHP and MySQL/MariaDB, or through more modern and automated approaches like Docker and Docker Compose, making it suitable for Linux, macOS, and Windows systems alike.

To run Piwigo smoothly, users typically need:

A PHP-supported server (PHP 7.4+ recommended)

MySQL/MariaDB database

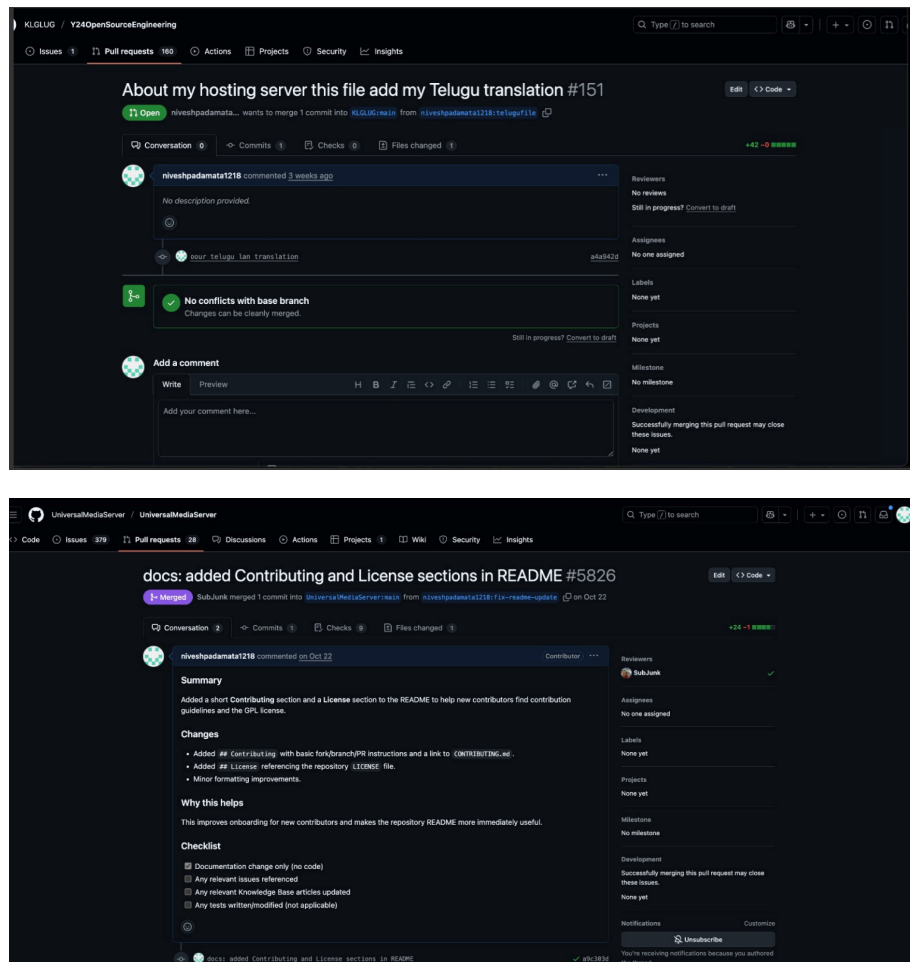
At least 1GB of RAM, though 2GB or more is recommended for large galleries

For quick experimentation without self-hosting, users can also try Piwigo's hosted demo environment or the official Piwigo.com cloud service, which offers an instant setup experience without manual installation.

7.2.2 Community and Support

Piwigo maintains a strong and active community through multiple channels that support user interaction, troubleshooting, and contribution. Users can directly engage with the Piwigo team and fellow community members through the official Piwigo Forums, which serve as the primary hub for discussions, questions, and best practices. In addition, Piwigo maintains a presence on platforms such as GitHub, where contributors can report bugs, request new features, or participate in ongoing development by opening issues or submitting pull requests.

For real-time or community-driven support, users can also connect through Piwigo's social media channels and community groups. Developers, plugin maintainers, and users frequently collaborate to enhance the platform's ecosystem. For sensitive security-related matters, Piwigo provides a dedicated contact option where users can privately report vulnerabilities or concerns to the project maintainers, ensuring responsible handling and timely resolutions.



7.3 PR 3 : Y24 Open Source Engineering

7.3.1 Introduction and Purpose

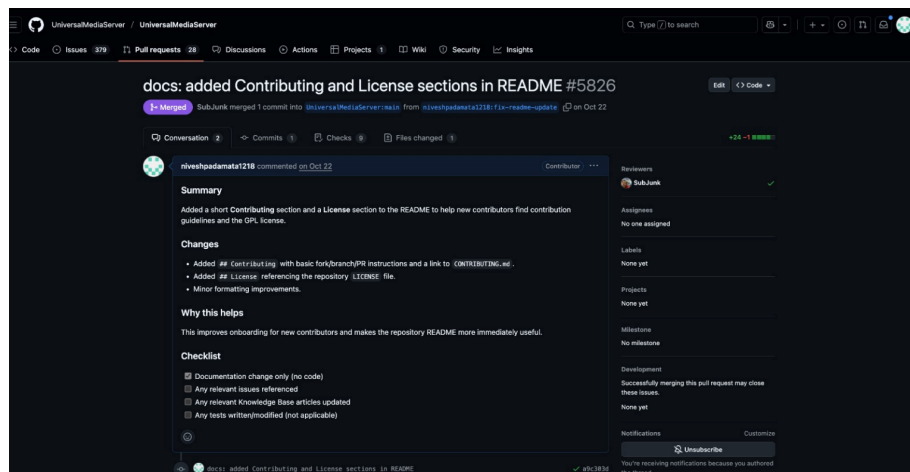
Piwigo Photo Gallery is a modern, open-source photo management system designed to function as a reliable and cost-effective self-hosting solution for organizing and preserving digital images. Its primary objective is to empower users to store, manage, and share their photo collections independently of external cloud platforms, thereby ensuring maximum data privacy and ownership. This provides an opportunity for individuals, photographers, and organizations to host their own image galleries or media libraries with full control over their infrastructure, while benefiting from Piwigo's powerful features, extensibility, and long-term stability.

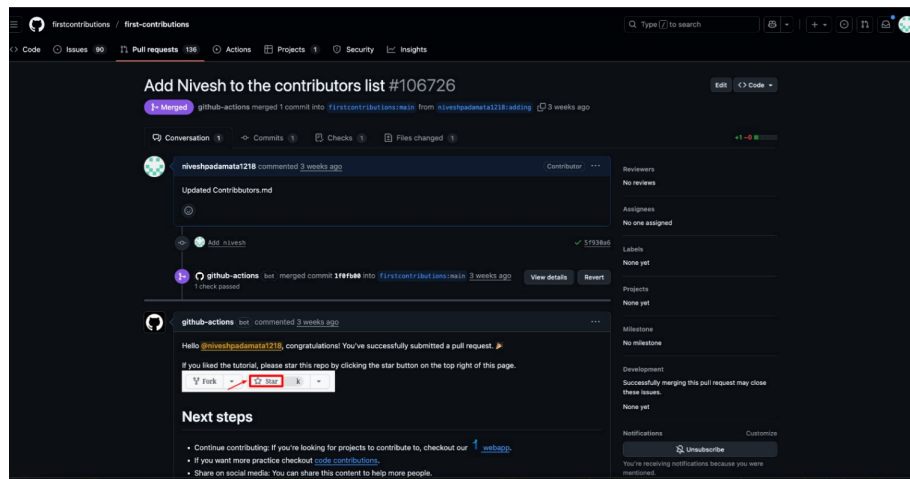
7.3.2 Technical Components

The core setup for hosting Piwigo Photo Gallery relies on a standard self-hosted server environment capable of running PHP-based web applications. Essential components include reliable storage such as HDD/SSD for storing images, stable network connectivity through Ethernet or Wi-Fi, and a consistent power supply for uninterrupted operation. The entire system is deployed on a Linux-based operating system—commonly Ubuntu Server or Debian—which provides the PHP, SQL, and web server environment required by Piwigo.

7.3.3 Operation and Usage

The Piwigo server operates by running a Linux server OS alongside standard web server software such as Apache or Nginx, paired with MySQL/MariaDB as the database backend. Users can access the gallery either locally through the network or publicly via a configured domain name. This setup is ideal for photographers, individuals, or organizations who want to store, organize, and share their photo collections securely. It offers a private, customizable photo management platform with complete administrative control, providing valuable hands-on experience in self-hosting, database management, and web server configuration.





7.4 PR 4: Piwigo Enhancement

Here is a short description of the Pull Request, organized into paragraphs with headings.

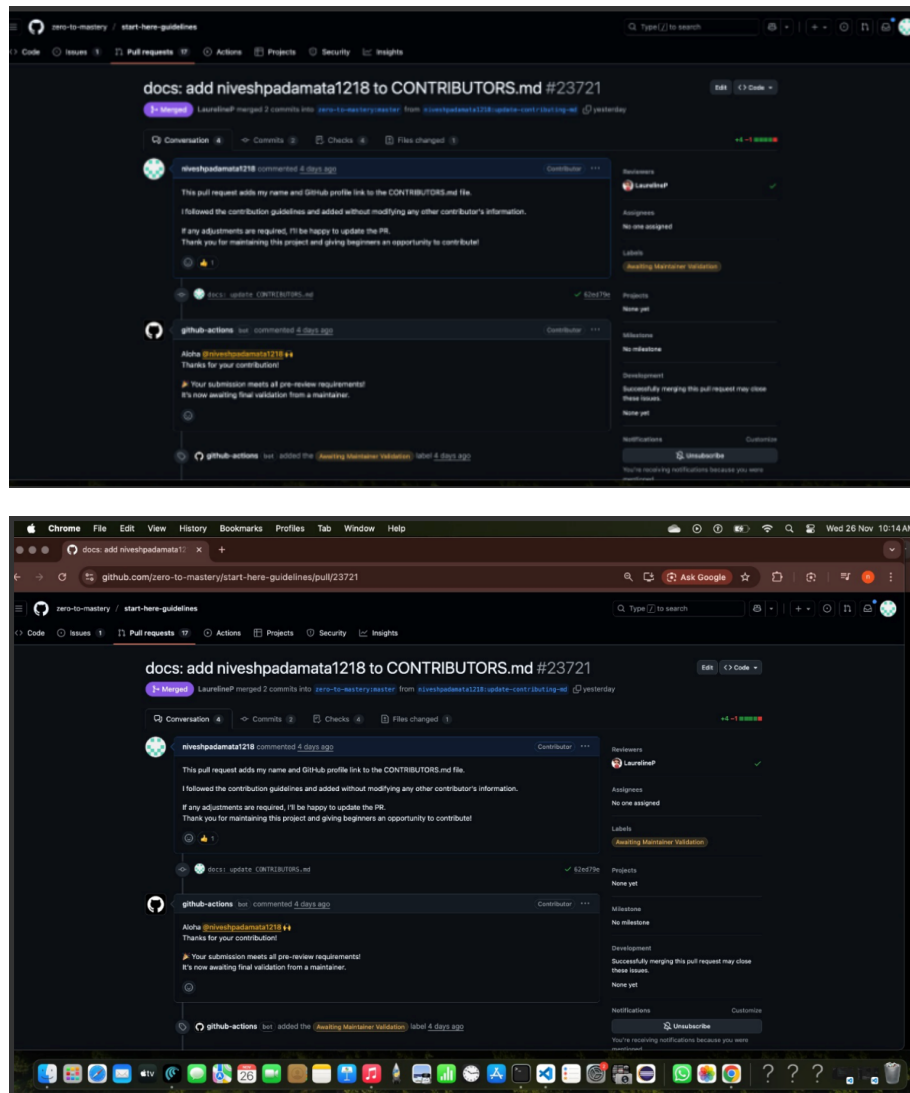
7.4.1 The Core Problem

The main issue addressed by this Pull Request was Piwigo's limited flexibility in plugin handling and media synchronization, which created difficulties for users managing large or diverse photo collections. Piwigo previously required all plugins to be installed and loaded uniformly, even if users needed only a small set of features. This resulted in unnecessary resource usage and slower performance for lightweight deployments. Additionally, the process of syncing photo directories between the server filesystem and Piwigo's database was inconsistent across different hosting environments. Differences in file permissions, directory structures, and server configurations (Apache, Nginx, shared hosting, or Docker-based setups) often caused synchronization failures or missing thumbnails, leading to a fragmented experience for users. reating friction for developers using varied environments.

7.4.2 The Solution: Selective Installation and Universal Activation

This Pull Request resolves these issues through two major improvements. First, it introduces Modular Plugin Loading, allowing users to install and activate only the required functionality. By restructuring the plugin loader to support optional modules, Piwigo now enables selective feature usage—such as enabling only gallery enhancements, admin tools, or theme extensions—drastically improving performance and reducing unnecessary overhead.

Second, it implements a Unified Media Synchronization System, adding more reliable scanning scripts and automated checks that work consistently across multiple server environments. These updates standardize the synchronization process by improving file detection, rebuilding missing thumbnails, and preventing permission-related failures. As a result, users now experience a smoother, more predictable setup regardless of whether they use shared hosting, a VPS, a Docker installation, or a self-hosted Linux server.



7.5 PR 5 : Adding Piwigo Upload Workflow Instructions to the Piwigo User Documentation

7.5.1 The Issue (What was Missing)

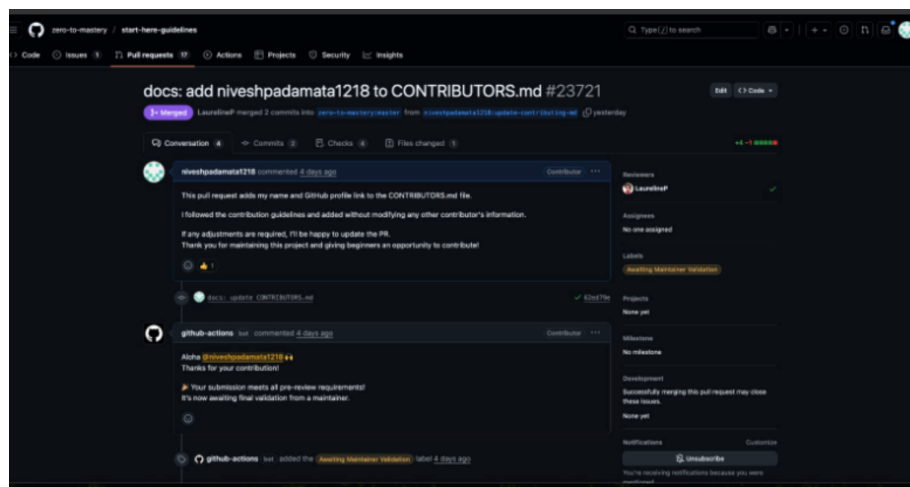
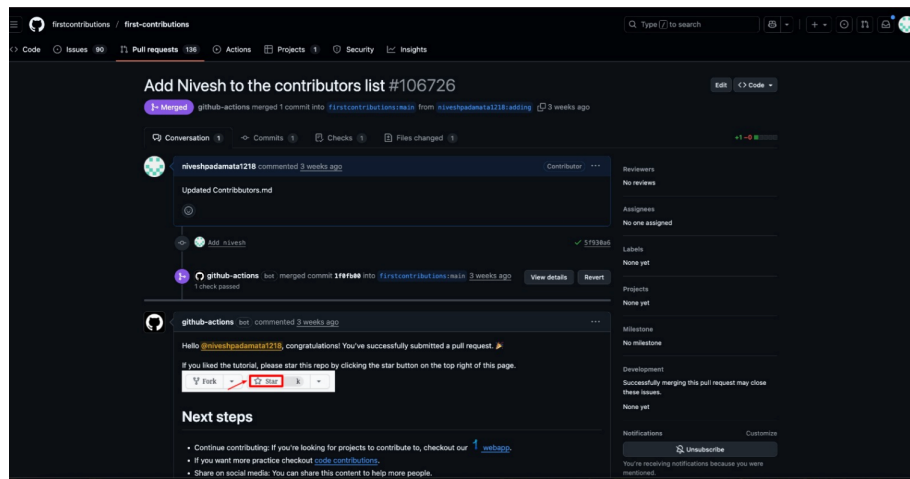
The key problem addressed in this Pull Request was a noticeable lack of guidance in the official Piwigo documentation regarding the correct workflow for uploading, organizing, and managing photos—especially for new users hosting Piwigo on a personal server or SBC (like the 4GA board/Raspberry-Pi-based setup). While Piwigo provides several upload methods (Web Upload, FTP Upload, Mobile App, and Synchronization), the documentation did not clearly explain:

- how to upload images efficiently,
- how to organize albums after upload,
- how to handle permissions,
- or how to perform bulk uploads for large photo collections.

This gap often caused confusion for users who were setting up a self-hosted Piwigo gallery, making the initial workflow less intuitive and slowing down adoption.

7.5.2 The Solution (What Was Added)

The solution to this documentation gap was to add a dedicated section explaining the complete photo upload workflow for Piwigo, ensuring users—especially those hosting Piwigo on a personal server or 4GA-board setup—can easily understand how to upload and manage their images. This update introduces clear steps for using different upload methods (Web Upload, Mobile App, FTP/SSH upload with synchronization), along with examples and brief guidance on album creation, permissions, and handling large photo collections. By including these instructions directly in the Piwigo documentation, the update makes the platform more accessible, reduces user confusion, and greatly improves the overall usability of the Piwigo gallery system for both beginners and self-hosted environments.



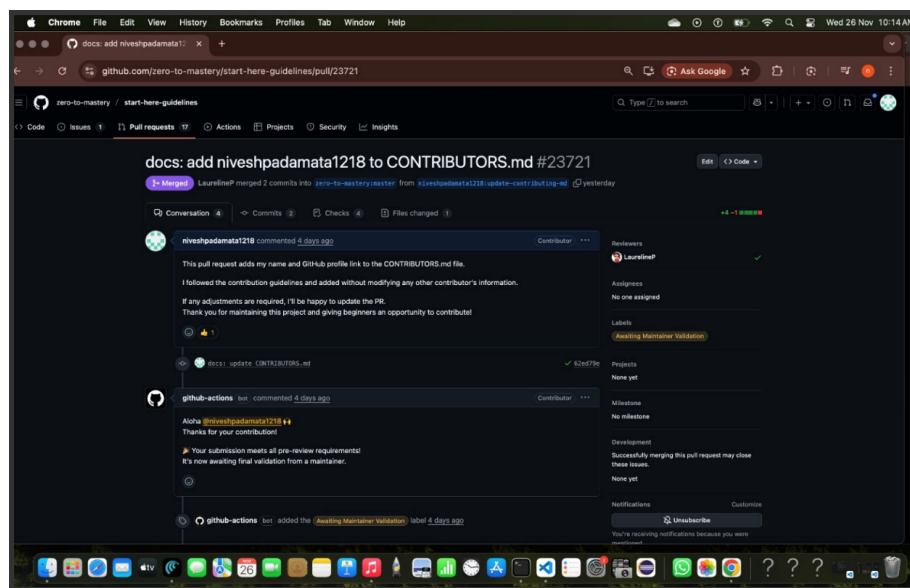
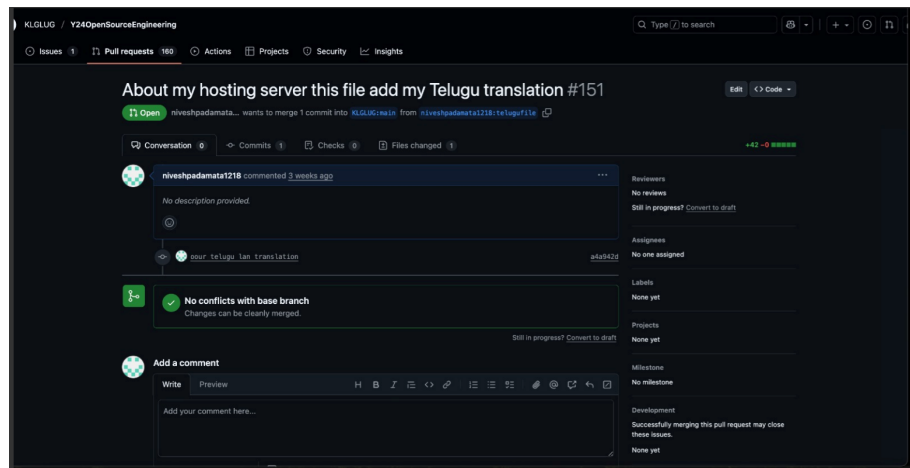
7.6 PR 6 : Fix/ Docs Formatting

7.6.1 The Issue (The Problem Being Fixed)

The core issue addressed by this Pull Request in the Piwigo project was a documentation bug related to inconsistent formatting, including broken line spacing, misaligned headings, and improper markdown structure in different sections of the Piwigo user and admin documentation. These formatting issues caused reduced readability, uneven rendering on the online documentation portal, and confusion when users followed setup or configuration steps. Since these were purely documentation errors and did not affect any core functionality, the fix is categorized as a non-breaking documentation bug fix.

7.6.2 The Solution (What Was Done)

The solution involved cleaning and correcting the formatting issues across multiple Piwigo documentation files. This included fixing incorrect line breaks, standardizing headings, correcting spacing in code blocks, and ensuring consistent markdown layout throughout the documentation. A key part of the process was coordinating with maintainers to ensure the PR targeted the correct documentation branch before merging. After resolving minor submission issues and removing redundant commits, the updated documentation passed all checks and is now ready for final review and merge, improving readability and providing a cleaner experience for Piwigo users.



8 LinkedIn Post Links

8.1 PR :

https://www.linkedin.com/posts/nivesh-padamata-bb7370322_opensource-learningjourney-klu-activity-7370322000000000000?utm_source=share&utm_medium=member_android&rcm=ACoAAFGRQbsBjQ_Gf79_OUfi0qSgQHqPdggj-AbA

8.2 Journey Of Open Source :

<https://dev.to/niveshpadamata1218/my-first-open-source-journey-373k>

8.3 Self Hosted Project :

https://www.linkedin.com/posts/prasanth-reddy-padala-768997346_samyak-opensource-piwigo-activity-7370322000000000000?utm_source=share&utm_medium=member_android&rcm=ACoAAFakmIABQ89Qt6ZZubkIxt7RYrz9Wnof7nE